AKHudfC Excel Add-in Background Notes

Document Purpose

The AKHudfC Excel Add-in was initially conceived and developed as a basic tool for assisting with the preparation of engineering calculations.

The purpose of this document is to record the general methodology and procedure used to initially develop the AKHudfC Excel Add-in. This document reflects the original (2023) setup notes and may be outdated. It is retained as background/reference, not as current installation instructions.

https://github.com/debear81/AKHudfC

General References

https://colinlegg.wordpress.com/2016/09/07/my-first-c-net-udf-using-excel-dna-and-visual-studio/

https://www.c-sharpcorner.com/article/transposeby-extending-excel-with-c-sharp-and-excel-dna/

https://github.com/Excel-DNA/IntelliSense

https://github.com/Excel-DNA/IntelliSense/wiki/Usage-Instructions

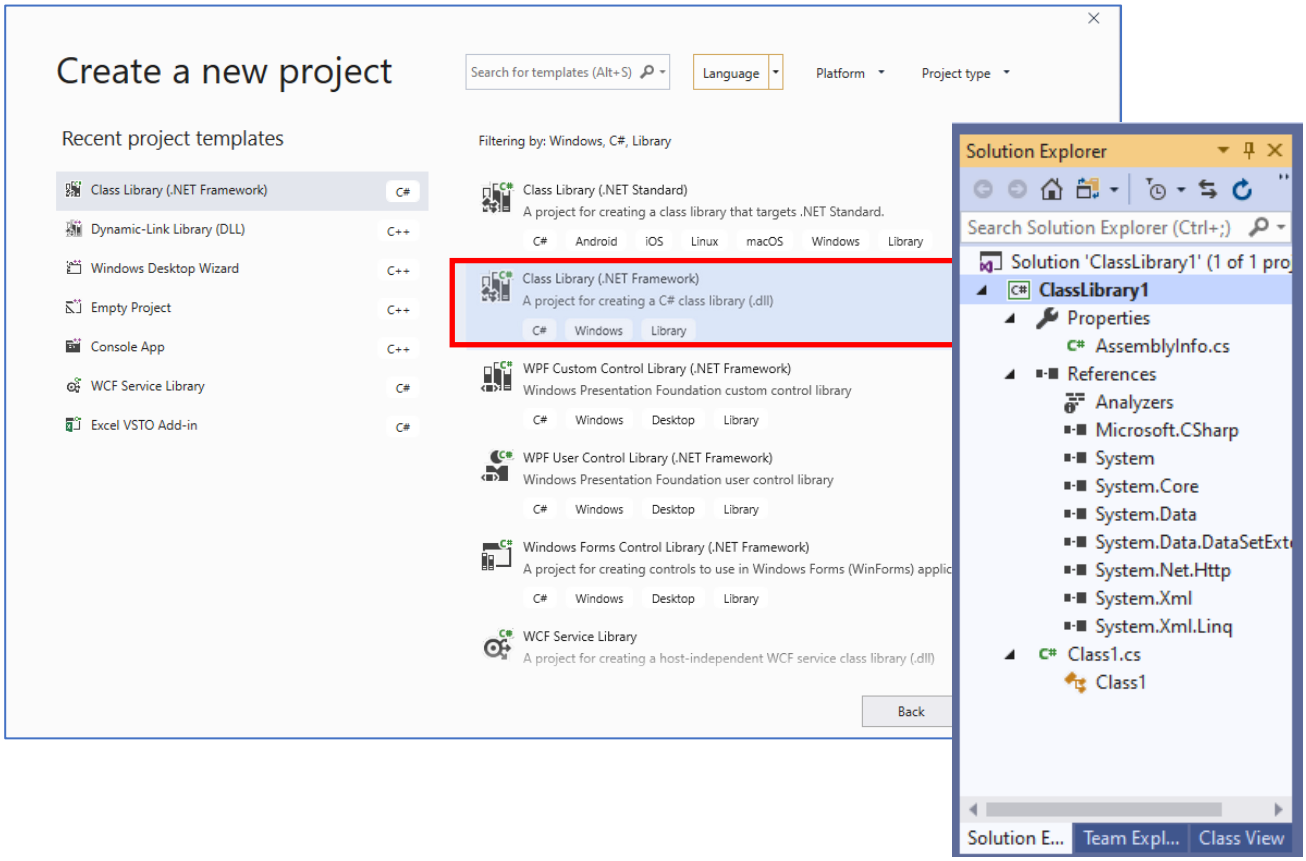
Software Configuration

This was the original project configuration used (2023). It may not be the most up to date.

Excel® for Microsoft 365 MSO (16.0.13929.20206) 32-bit
Microsoft Visual Studio Community 2022, Version 16.2.2
.NET Framework 4.8
~~C# Tools 3.2.0 (not sure what this was, June 2023)~~
NuGet Package Manager 5.2.0
~~(ExcelDNA v. 1.6.0 seemed to be causing weird errors as of June 2023)~~
ExcelDNA v. 1.5.1 https://excel-dna.net/
    Intellisense https://github.com/Excel-DNA/IntelliSense
    Registration Helper https://github.com/Excel-DNA/Registration

MS Visual Studio Project Creation

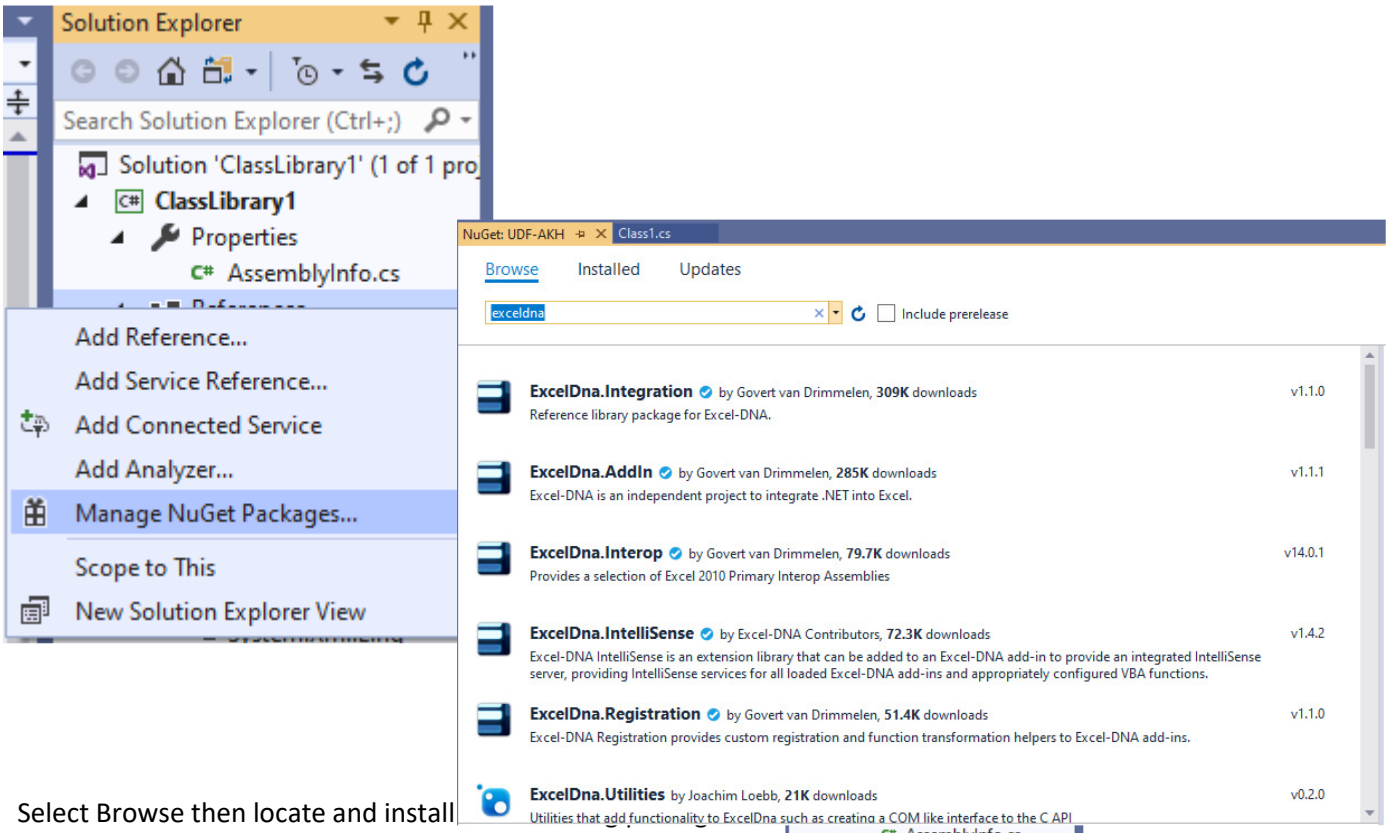
Create a new C# class library for .NET framework project in Visual Studio.



Visual Studio will create a new solution (project) and populate it with the required general files and information.

Install/Reference ExcelDNA

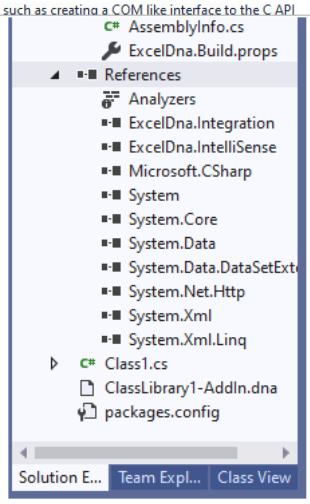
If you don't feel like installing from the **Package Manager Console**, in the **Solution Explorer**, right-click on **References** and select **Manage NuGet Packages…**



Select Browse then locate and install

- ExcelDNA.AddIn
  (v. 1.5.1 at time of writing)
- ExcelDNA.Integration
  (v. 1.5.1 at time of writing)
- ExcelDNA.Intellisense
  (Optional, v. 1.5.1 at time of writing)

Then NuGet will add the required files and references.



For creating and editing an addin, ExcelDNA requires you to have .Net Framework Developer Pack 4.5 (or 4.8) installed.

Excel UDF General Creation

Open or create an empty *.cs and add the code that will be used to create a User Defined Function for Excel.

```
using System;
using System.Globalization;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using ExcelDna.Integration;

// note that this namespace is split across multiple CS files
namespace UDFakh
{
    // ==================== START of Class ====================
    public class FuncString : XlCall
    {
        // ==================== START of Function ==================
        [ExcelFunction(Description ="A useful test function that adds two numbers.")]
        public static double AddThem
            ([ExcelArgument(Name ="Augend",Description ="is the first number ")]
            double v1,
            [ExcelArgument(Name ="Addend", Description ="is the second number that will be added")]
            double v2
            )
                { return v1 + v2;
                }
    } // ========== END Class ==========

} // ========== END Namespace ==========
```

ExcelDNA Intellisense

ExcelDNA has included the use of Intellisense to provide on-screen tooltips for UDFs similar to what is included for that native Excel functions.

The required “Name =” and “Description =” have been included in the (cs) text above, but further steps need to be taken for Excel to recognize these.

https://github.com/Excel-DNA/IntelliSense/wiki/Usage-Instructions

Add the following lines of code to the top of the function definition (*.cs) file. This will add the descriptions when the UDFs are inserted to an Excel cell via the function button, but it will not show up on screen.

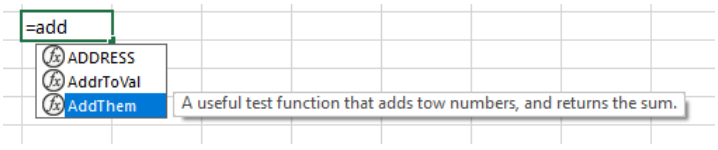
- *using ExcelDna.Integration;*
- *using ExcelDna.IntelliSense;*
-

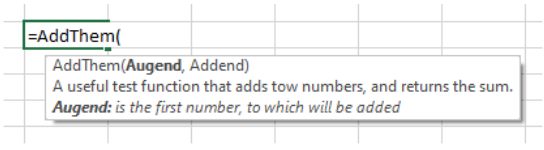
If you want to enable the On-Sheet tool-tips and descriptions for your UDF, you need to set up an AutoOpen handler to load / install the IntelliSenseServer. This can be done in the function definition (***.cs)** file that was previously created. Or it can be set up in a separate *.cs file, that uses the same namespace as the function definition file.

```
using ExcelDna.Integration;
using ExcelDna.IntelliSense;

// note that this namespace is split across multiple CS files
namespace UDFakh
{
    // In order for the on-sheet Tool-Tips and descriptions to work, you need to
    // Register the IntelliSenseServer in your add-in's AutoOpen() implementation
    // https://github.com/Excel-DNA/IntelliSense/wiki/Usage-Instructions
    public class Auto : IExcelAddIn   // this is a Class inheritance, where "Auto" is the child
    {
        public void AutoOpen()
        {
            IntelliSenseServer.Install();
        }

        public void AutoClose()
        {
            IntelliSenseServer.Uninstall();
        }
    }
}
```





Variable Number of Function Parameters / Arguments

As of 2017-ish, ExcelDNA can handle variable # of parameters (C# "paramArrayAttribute"). The Excel-DNA Registration extension supports params registration and other type conversions

https://github.com/Excel-DNA/Registration

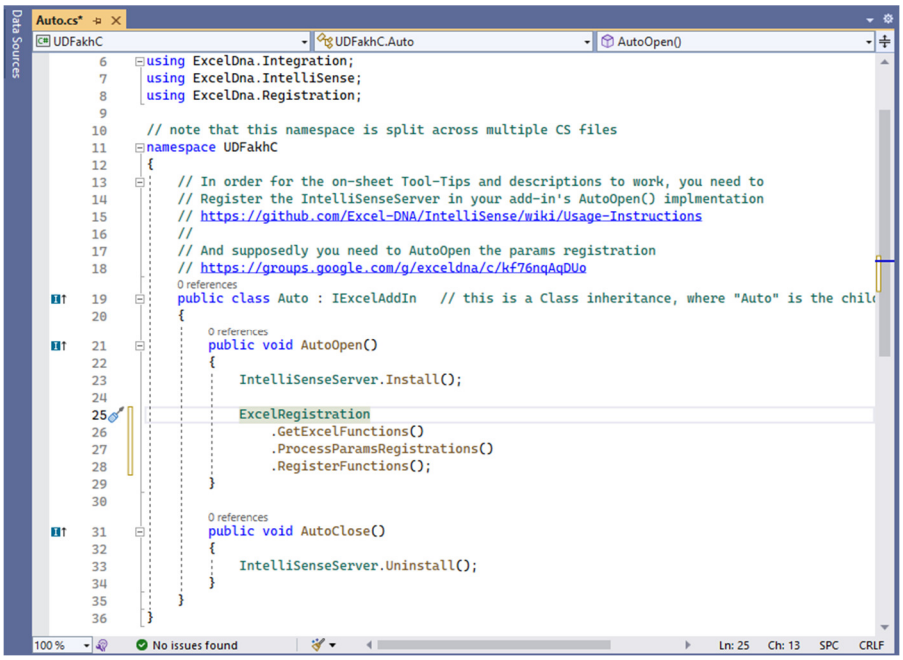
https://groups.google.com/g/exceldna/c/kf76nqAqDUo

Configure the project for Explicit Registration by editing the *add-in project name*.dna by adding the following:

<ExternalLibrary Path="*name*.dll" ExplicitRegistration="true" LoadFromBytes="true" Pack="true" />

Add the registration processing code in an AutoOpen() implementation:

```
public void AutoOpen()
  {
    ExcelRegistration
      .GetExcelFunctions()
      .ProcessParamsRegistrations()
      .RegisterFunctions();
  }
```



Functions with Optional Arguments

In VBA, you can create a UDF that uses optional arguments (or arguments that use a default value, when no value is entered).

However, at present, there is no inherent convention for optional arguments when creating a UDF with ExcelDNA. Though the smart folks working on the ExcelDNA project do offer a workaround to allow for using optional arguments in your UDFs.

https://docs.excel-dna.net/optional-parameters-and-default-values/

They propose defining your optional argument as an object in the function definition and and converting it to the required data type with the helper class.

Snippet of function code:

```
        [ExcelFunction(Description = "A speed test function that counts from 0 to the BigNumber,
using Step as increment.")]
        public static string BiggerNumber
            ([ExcelArgument(Name="BigNumber",Description ="The number that will be counted to.")]
              double NumbDbl,
             [ExcelArgument(Name ="Step",Description ="step")]
              object StepArg // optional int argument
            )
            {
                {   // assign default value to optional argument, see "OptionalArg" helper class
                int Step = OptionalArg.Check(StepArg, 1);…
```

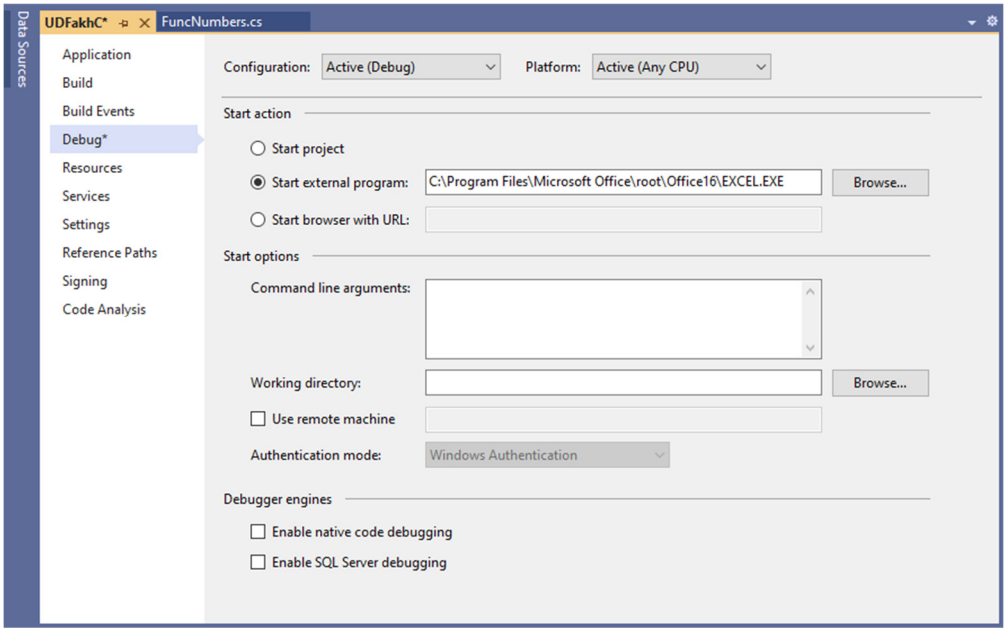
Snippet of helper Class code:

```
        internal static string Check(object arg, string defaultValue)
        {
            if (arg is string)
                return (string)arg;
            else if (arg is ExcelMissing)
                return defaultValue;
            else
                return arg.ToString();  // Or whatever you want to do here....
        }

        internal static int Check(object arg, int defaultValue)
        {
            if (arg is int)
                return (int)arg;
            else if (arg is ExcelMissing)
                return defaultValue;
            else
            {
                throw new ArgumentException(); // Will return #VALUE to Excel
            }
        }
```

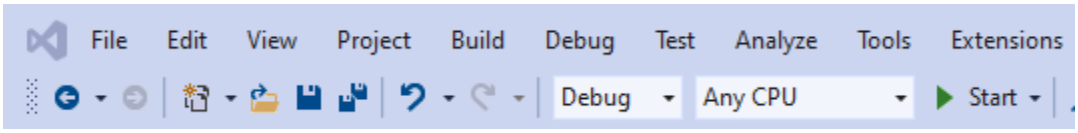
Start Action for Excel

Make sure the “Start external program” option is checked in the Debug properties of the project.



Compiling & Debugging

If you don’t have any code errors in the **Error List**, you can just **Start** the Debug process.

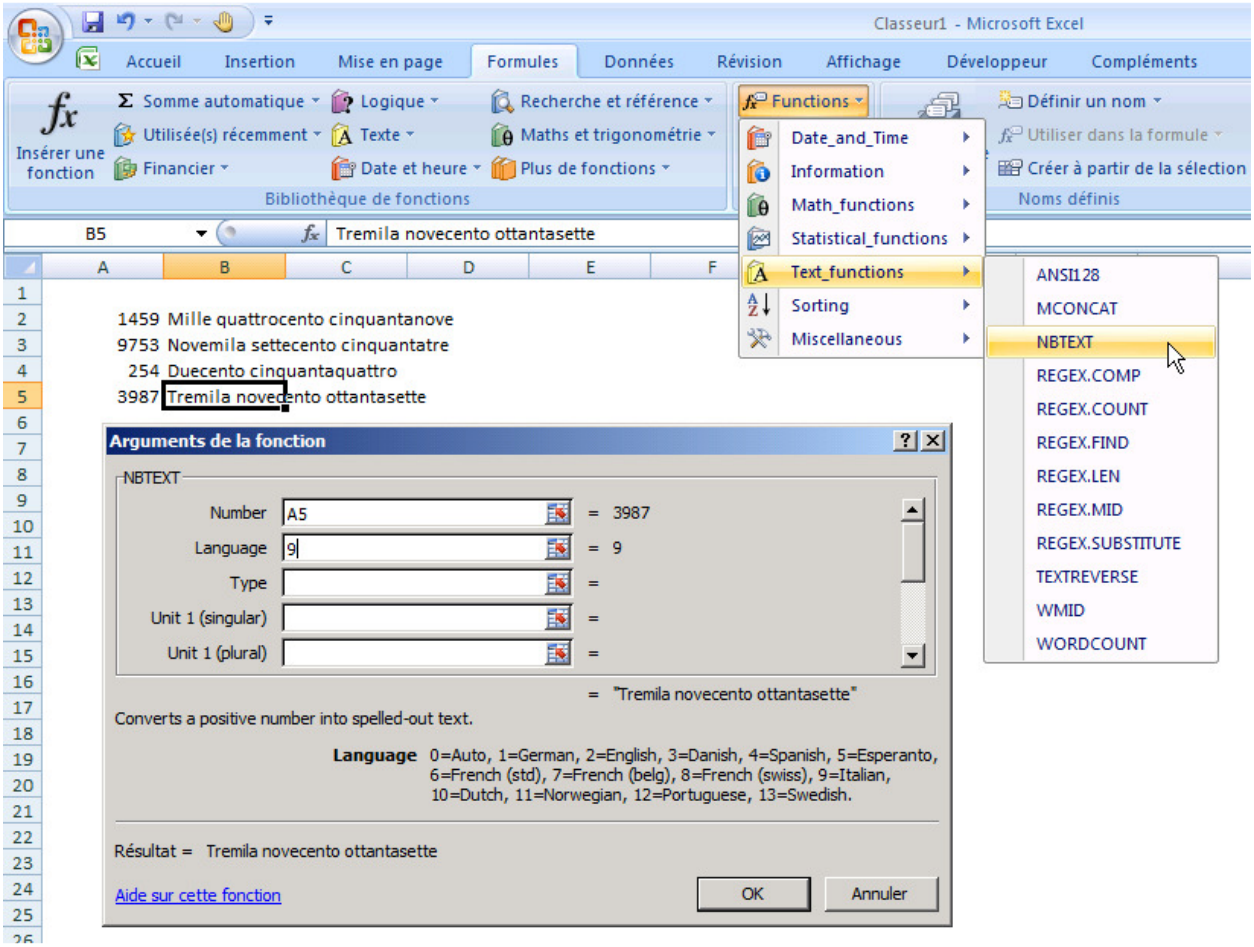


At this point, Excel should automatically open. Start a new workbook and test out the UDF.

Background

Several years ago, I stumbled over a very useful Excel add-in, MOREFUNC, that was created in C++ by Laurent Longre. It contained several additional Excel functions that covered topics in Text manipulation, Statistical functions, and several others. Sadly, development and support appeared to cease sometime around 2007. And, to the best of my knowledge, it stopped functioning once the 64-bit version of Excel was introduced.

This add-in effort is not intended as a replacement for MOREFUNC. MOREFUNC was merely a mental seed.



http://web.archive.org/web/20111120083730/http://xcell05.free.fr/morefunc/english/index.htm